APCS 程式設計系列 · U11 · 實作高級 · C++版

動態規劃 詳解

遞迴 → 記憶化 → 由下而上 · DP 四步驟 · 爬樓梯與方格路徑 · 最大連續和 · 0/1 背包 · LIS 與 LCS

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
A	B	C	D	A	B	C	D	A	B
11	**12**	**13**	**14**						
C	D	A	B						

Part A 程式識讀詳解

第 1 題 答案：(A)

- 由小到大一格一格填：f[2] = 1 + 2 = 3，f[3] = 3 + 2 = 5，f[4] = 5 + 6 = 11。
- f[5] = 11 + 10 = 21，f[6] = 21 + 22 = 43。

【錯】f[5] = 21 是前一格，看錯索引就會選 21。

第 2 題 答案：(B)

- 設 c(n) 為呼叫 f(n) 總共會進入函式幾次：c(0) = c(1) = 1，c(n) = 1 + c(n-1) + c(n-2)。
- c(2) = 3，c(3) = 5，c(4) = 9，c(5) = 1 + 9 + 5 = 15。

【錯】f(5) 的「值」是 5，但題目問的是呼叫次數。這也說明純遞迴的重複計算有多嚴重。

第 3 題 答案：(C)

- d[3] = 2 + 1 + 1 = 4，d[4] = 4 + 2 + 1 = 7。
- d[5] = 7 + 4 + 2 = 13，d[6] = 13 + 7 + 4 = 24。

第 4 題 答案：(D)

- 第 0 列：1 1 1 1；第 1 列：1 0 1 2；第 2 列：1 1 2 4。
- 也可以這樣想：沒有障礙共 10 條，經過障礙的有 2 × 3 = 6 條，10 - 6 = 4。

第 5 題 答案：(A)

- cur 依序是 -2、1、-2、4、3、5、6、1、5。
- best 取過程中的最大值 6（區段 4, -1, 2, 1）。

【錯】最後的 cur 是 5，只看最後一格就會選 5。

第 6 題 答案：(B)

- cur 每次加上負數就被歸零，best 從 0 開始，從來沒有被更新。
- 所以印出 0。

【錯】如果題目規定「至少選一個數」，正確答案應該是 -1；初始值設成 0 是常見錯誤。

第 7 題 答案：(C)

- 容量倒著跑，是 0/1 背包：每個物品最多拿一次。
- 兩個物品都拿：重量 5、價值 3 + 5 = 8。

第 8 題 答案：(D)

- 正著跑時同一個物品可以重複拿，變成無限背包。
- 重量 2 拿兩次、重量 3 拿一次：重量 7、價值 3 + 3 + 5 = 11。

【錯】以為結果跟倒著跑一樣是 8。

第 9 題 答案：(A)

- d 依序是 1、1、2、1、3、4、2、4。
- d[5]（數字 9）= 4：例如 3, 4, 5, 9；d[6]（數字 2）只能接在 1 後面，= 2。

第 10 題 答案：(B)

- d[4][3] 是 "ABCB" 與 "BDC" 的 LCS："BC"，長度 2。
- d[n][m] 是整個字串的 LCS，例如 "BCBA"，長度 4。

第 11 題 答案：(C)

- 倒著跑時，算 dp[c] 用到的 dp[c - w] 比 c 小，這一輪還沒被更新，代表「還沒拿這個物品」。
- 正著跑的話，dp[c - w] 可能已經拿過這個物品，等於同一個物品被拿好幾次。

第 12 題 答案：(D)

- 每一步都取模，等於只保留個位數；結果和「算完再取模」相同。
- 費氏數列第 20 項是 6765，個位數是 5。

第 13 題 答案：(A)

- dp[2][5]：只用前兩個物品（重 1 價 1、重 3 價 4），容量 5，兩個都拿：價值 5。
- dp[3][7]：拿重 3 和重 4 的物品，重量剛好 7，價值 4 + 5 = 9。

【錯】三個全拿價值是 10，但重量 8 超過容量 7。

第 14 題 答案：(B)

- memset 是「每一個位元組」都設成同一個值。-1 的每個位元組都是 0xFF，所以 int 剛好是 -1。
- 設成 1 時，int 的 4 個位元組都是 0x01，等於 0x01010101 = 16843009。

【錯】記憶化初始化只用 memset 設 0 或 -1；其他值要用迴圈。

Part B 程式實作參考解答

實作 1 壞掉的樓梯

解題思路

- 狀態：dp[i] = 剛好走到第 i 階的走法數。
- 轉移：最後一步是從i-1、i-2或i-3階上來的，dp[i]=dp[i-1]+dp[i-2]+dp[i-3]（超出範圍的不算）。
- 壞掉的階 dp[i] = 0；初始值 dp[0] = 1；從 1 算到 n，答案 dp[n]。

【錯】初始值 dp[0] 要是 1（站在地面也算一種），設成 0 會讓答案全部變 0。
【錯】每次相加後都要取模；三個數相加最大約 3×10^9，C++ 要用 long long 存。
【錯】n 到 10^6，寫成遞迴會太深；用陣列由下而上算。

C++ 壞掉的樓梯

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<bool> bad(n + 1, false);
    for (int k = 0; k < m; k++) {
        int x;
        cin >> x;
        bad[x] = true;
    }
    vector<long long> dp(n + 1, 0);
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        if (bad[i]) continue;
        long long s = dp[i - 1];
        if (i >= 2) s += dp[i - 2];
        if (i >= 3) s += dp[i - 3];
        dp[i] = s % MOD;
    }
    cout << dp[n] << "\n";
}
```

時間複雜度：O(n)

實作 2 避開水窪

解題思路

- 狀態：dp[i][j] = 走到 (i, j) 的方法數；轉移：水窪格是 0，其他格 = 上面 + 左邊。
- 計算順序：一列一列、由左到右，算到 (i, j) 時，上面和左邊都已經算好。
- 空間可以壓成一維：dp[j] 在更新前是「上面那格」，dp[j-1] 是「左邊那格」（已經更新過），這叫滾動陣列。

【錯】第一列、第一行不能直接全設 1：中間有水窪時，水窪後面的格子走不到。

【錯】起點是水窪時要輸出 0，不能先把起點設成 1 就不管了。

【錯】相加後記得取模；用 long long 或 int 都放得下兩個小於 MOD 的數相加。

C++ 避開水窪

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int r, c;
    cin >> r >> c;
    vector<long long> dp(c, 0);
    dp[0] = 1;
    for (int i = 0; i < r; i++) {
        string row;
        cin >> row;
        for (int j = 0; j < c; j++) {
            if (row[j] == '#') dp[j] = 0;
            else if (j > 0) dp[j] = (dp[j] + dp[j - 1]) % MOD;
        }
    }
    cout << dp[c - 1] << "\n";
}
```

時間複雜度：O(RC)

實作 3 可以跳過一天的報告

解題思路

- 用兩個狀態：keep = 以第 i 天結尾、還沒拿掉任何一天的最大和；skip = 以第 i 天結尾、已經拿掉一天的最大和。
- keep 的轉移和最大連續和一樣：max(前一天的 keep + a_i, a_i)。
- skip 有兩種來源：拿掉的是今天 → 前一天的 keep；之前已經拿掉 → 前一天的 skip + a_i。
- 初始值：第 1 天 keep = a_1，skip 不存在（設成極小值）。答案是所有 keep、skip 中的最大值。

【錯】初始值用 0：全部是負數時會錯印 0（範例 2 的答案是 -2）。

【錯】skip 一開始不能設 0：只有一天時拿掉它就沒東西了，報告至少要留一天。

【錯】總和到 2×10^{14}，要用 long long；極小值用 LLONG_MIN / 4，避免再加上負數時溢位。

C++ 可以跳過一天的報告

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    const long long NEG = LLONG_MIN / 4;
    long long keep, skip = NEG, ans;
    cin >> keep;
    ans = keep;
    for (int i = 1; i < n; i++) {
        long long x;
        cin >> x;
        long long nskip = max(skip + x, keep);   // 之前拿掉 / 拿掉今天
        long long nkeep = max(keep + x, x);
        keep = nkeep;
        skip = nskip;
        ans = max(ans, max(keep, skip));
    }
    cout << ans << "\n";
}
```

時間複雜度：O(n)

實作 4 畢業旅行的背包

解題思路

- 這是標準的 0/1 背包。狀態：dp[c] = 容量 c 能得到的最大快樂值。
- 一件一件考慮：對每件東西，容量 c 從 W 倒著跑到 w_i，dp[c] = max(dp[c], dp[c - w_i] + v_i)。
- 初始值全部是 0（什麼都不帶）；答案在 dp[W]。
- n × W 最多 2×10^6，O(nW) 跑得完。

【錯】容量迴圈正著跑會變成無限背包：測資「1 10 / 1 7」正確答案 7，正著跑會得到 70。
【錯】快樂值總和最大約 10^{11}，dp 要用 long long。
【錯】重量比 W 大的東西帶不進去，迴圈自然不會執行，不用特別處理。

C++ 畢業旅行的背包

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, W;
    cin >> n >> W;
    vector<long long> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int w;
        long long v;
        cin >> w >> v;
        for (int c = W; c >= w; c--)          // 倒著跑
            dp[c] = max(dp[c], dp[c - w] + v);
    }
    cout << dp[W] << "\n";
}
```

時間複雜度：O(nW)

實作 5 夜市美食路線

解題思路

- 這是 LIS 的變形：長度換成美味度總和。狀態：dp[i] = 最後吃第 i 攤時的最大美味度總和。
- 轉移：往前找所有 $j < i$ 且 $h_j < h_i$，$dp[i] = \max(dp[j]) + s_i$；找不到就是 s_i 自己。
- 答案是所有 dp[i] 的最大值（最後一攤不一定要吃）。
- $n \leqq 2000$，$O(n^2)$ 約 2×10^6 次比較，跑得完。

【錯】「嚴格遞增」要用 <，寫成 <= 會讓相同高度的攤位都吃（範例 2 會錯成 18）。
【錯】答案輸出 dp[n-1] 是錯的：範例 1 吃到第 4 攤（高度 1，美味度 100）再吃第 5 攤才是最佳。
【錯】總和最大 2×10^{12}，要用 long long。

C++ 夜市美食路線

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> h(n), s(n), dp(n);
    for (auto &x : h) cin >> x;
    for (auto &x : s) cin >> x;
    long long ans = 0;
    for (int i = 0; i < n; i++) {
        long long best = 0;
        for (int j = 0; j < i; j++)
            if (h[j] < h[i] && dp[j] > best) best = dp[j];
        dp[i] = best + s[i];
        ans = max(ans, dp[i]);
    }
    cout << ans << "\n";
}
```

時間複雜度：$O(n^2)$

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
遞迴與記憶化	A2、A14		綠 黃 紅	講義第 1 段
DP 四步驟與取模	A1、A3、A12、B1		綠 黃 紅	講義第 2 段
方格路徑	A4、B2		綠 黃 紅	講義第 3 段
最大連續和	A5、A6、B3		綠 黃 紅	講義第 4 段
0/1 背包	A7、A8、A11、A13、B4		綠 黃 紅	講義第 5 段（一維要倒著跑）
LIS	A9、B5		綠 黃 紅	講義第 6 段
LCS	A10		綠 黃 紅	講義第 7 段